Algorithms

Lecture 2: Binary Search

Demo Lecturer

Semester 2, 2026

Last Lecture

- Cost is measured by counting the **basic operation**
- $t(n) \in O(g(n))$ iff $t(n) \leq c \cdot g(n)$ for all $n > n_0$
- Linear search: worst case $\Theta(n)$ comparisons

Today: can we do better if the array is sorted?

The Problem: Searching a Sorted Array

Input an array $A[0..n-1]$ sorted in increasing order, and a key x

Output an index j with $A[j] = x$, or -1 if x is not in A

Linear search ignores the order and may scan everything.
The order should let us *rule out many positions with one comparison*.

The Idea: Halve the Search Space

Guess a number between 1 and 100; after each guess you are told "higher" or "lower".

Guesses so far	Candidates left (at most)
0	100
1	50
2	25
3	12
k	$\approx 100/2^k$

Binary Search: Pseudocode

```
function BinSearch(A, x, n)
    lo <- 0
    hi <- n - 1
    while lo <= hi
        mid <- floor((lo + hi) / 2)
        if A[mid] = x
            return mid
        else if A[mid] < x
            lo <- mid + 1
        else
            hi <- mid - 1
    return -1
```

Trace: Search for 23

$A = [2, 5, 8, 12, 16, 23, 38, 56, 72, 91], \quad x = 23$

Round	lo	hi	mid	A[mid]	Action
1	0	9	4	16	$16 < 23$, so lo $\leftarrow 5$
2	5	9	7	56	$56 > 23$, so hi $\leftarrow 6$
3	5	6	5	23	found, return 5

Why Must A Be Sorted?

The loop maintains an **invariant**:

Invariant

If x occurs in A, then it occurs in $A[\text{lo..hi}]$.

- $A[\text{mid}] < x$: sortedness means everything at or left of mid is $< x$, so discarding it is safe
- On an unsorted array the same discard can throw away x

Analysis

Basic operation: the comparison $A[\text{mid}]$ against x.

- **Best case:** x is at the first midpoint, so 1 comparison
- **Worst case:** the range halves every round; let $C(n)$ be the comparisons:

$$C(n) = C(\lfloor n/2 \rfloor) + 1, \quad C(1) = 1 \quad \Longrightarrow \quad C(n) = \lfloor \log_2 n \rfloor + 1 \in \Theta(\log n)$$

Summary and Homework

- Binary search needs **sorted** input and runs in $\Theta(\log n)$ in the worst case
- Common bugs: wrong loop condition ($<$ vs $\leq$), and forgetting the ± 1 when updating lo / hi

Homework. Explain why binary search fails on an unsorted array, with a concrete array of length 5 where it returns -1 although x is present.